

LC 1277 - Count Square Submatrices with All Ones

Authored by Arpit Raj, LNMIIT Jaipur | July 9, 2026

1. Bruteforce Approach

The brute-force method relies on the **Prefix Sum** combined with a row-column offset technique. We calculate the 2D prefix sums of the matrix to query the sum of elements within any submatrix in $O(1)$ time.

- Iterate over all possible top-left corners of a square.
- Iterate over all possible side lengths (len).
- Calculate the sum of the square using the prefix sum matrix.
- If the sum equals $len \times len$, it means the square is entirely composed of 1s, and we increment our answer.

Time Complexity: $O(N \times M \times \min(N, M))$

Space Complexity: $O(N \times M)$ for the prefix sum matrix.

```

class Solution {
public:
    int countSquares(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        vector<vector<int>> pref(n, vector<int>(m));

        for(int i=0; i<n; i++){
            for(int j=0; j<m; j++){
                pref[i][j] = matrix[i][j];

                if(i>0){
                    pref[i][j] += pref[i-1][j];
                }
                if(j>0){
                    pref[i][j] += pref[i][j-1];
                }
                if(i>0 && j>0){
                    pref[i][j] -= pref[i-1][j-1];
                }
            }
        }

        int ans = 0;
        for(int len=1; len<=min(n,m); len++){
            for(int roff=0; roff+len<=n; roff++){
                for(int coff=0; coff+len<=m; coff++){
                    int r1 = roff;
                    int c1 = coff;
                    int r2 = roff+len-1;
                    int c2 = coff+len-1;

                    int sum = pref[r2][c2];

                    if(r1>0){
                        sum -= pref[r1-1][c2];
                    }
                    if(c1>0){
                        sum -= pref[r2][c1-1];
                    }
                    if(r1>0 && c1>0){
                        sum += pref[r1-1][c1-1];
                    }

                    if(sum == len*len){
                        ans++;
                    }
                }
            }
        }
    }
};

```

```

        }
    }
}
return ans;
}
};

```

2. Better Approach (Histogram Method)

Instead of repeatedly summing matrices, we can view the problem identically to finding areas in a histogram. We convert each row into a histogram base, where the height represents the consecutive 1s looking upwards.

- **Build Histogram:** If the current cell is 0, height is 0. Else, height is $1 + \text{height}(\text{above})$.
- **Process:** For each row, fix the right boundary (r) and vary the left boundary (l). Keep track of the minimum height (minh) in that range.
- The largest square ending at the bottom-right for this width is bounded by the minimum height or the width itself: $\min(\text{minh}, \text{width})$.

Time Complexity: $O(N \times M^2)$

Space Complexity: $O(N \times M)$

```

class Solution {
public:
    int countSquares(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        int ans = 0;

        // Build Histogram
        vector<vector<int>> his(n, vector<int>(m));

        for(int i=0; i<n; i++){
            for(int j=0; j<m; j++){
                if(matrix[i][j] == 0){
                    his[i][j] = 0;
                }
                else{
                    if(i == 0){
                        his[i][j] = 1;
                    }
                    else{
                        his[i][j] = his[i-1][j] + 1;
                    }
                }
            }
        }

        // Process every histogram row
        for(int i=0; i<n; i++){
            for(int r=0; r<m; r++){
                int minh = INT_MAX;
                for(int l=r; l>=0; l--){
                    // Minimum height in current width
                    minh = min(minh, his[i][l]);
                    int width = r - l + 1;

                    // Largest square for this width
                    ans += min(minh, width);
                }
            }
        }
        return ans;
    }
};

```

3. Optimal Approach (Dynamic Programming)

Instead of asking how many squares end at a specific row, or processing entire rows at a time, we shift our perspective to individual cells. The core question becomes: **"How many squares end at this specific cell?"**

Defining the DP State

Let $dp[i][j]$ = the side length of the **largest square** whose bottom-right corner is at (i, j) .

Key Observations

- For a cell $(0,0)$, if it has a 1, it can only form a 1x1 square. So, $dp[0][0] = 1$.
- For the **entire first row and first column**, there is no room for a bigger square to expand upwards or leftwards. Hence, if $matrix[i][j] == 1$, then $dp[i][j] = 1$.
- To extend a square by one, you need support from the top, left, and diagonal (top-left) adjacent cells.

The minimum of these three neighboring values determines the maximum size we can "inherit". If all three neighbors can support a square of size X , the current cell can form a square of size $X + 1$.

Transition Equation:

If $matrix[i][j] == 0 \rightarrow dp[i][j] = 0$

Else $\rightarrow dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

Why do we simply add $dp[i][j]$ to the answer?

If a square of size 4 ends at a cell, it inherently implies that squares of sizes 1x1, 2x2, 3x3, and 4x4 also end at that exact same bottom-right corner. Therefore, the size of the maximum square directly equals the count of all squares ending there.

```
class Solution {
public:
    int countSquares(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        int ans = 0;
        vector<vector<int>> dp(n, vector<int>(m));

        for(int i=0; i<n; i++){
            for(int j=0; j<m; j++){
                if(matrix[i][j] == 0){
                    dp[i][j] = 0;
                }
                else if(i == 0 || j == 0){
                    dp[i][j] = 1;
                }
                else{
                    dp[i][j] = 1 + min(dp[i-1][j], min(dp[i][j-1], dp[i-1][j-1]));
                }

                ans += dp[i][j];
            }
        }

        return ans;
    }
};
```